Lab 10 Security logs and detection metrics

AI for IT Security Professionals | TGS-2023039344 | v6.0

Purpose and output

Create a time-window detection output and tuning note. This lab supports LO4. It uses a simplified deterministic teaching model, not a trained AI system or full Azure emulator. The AI review is a separate exercise using the included prompt.

Requirements

Python 3.10 or later, a text editor and this entire folder. No packages, network access or API key are required for the baseline. On Windows use py -3 if python3 is unavailable. An approved AI tool is optional; the trainer can provide a review response for critique. Azure exercises require the trainer-assigned subscription, permission and feature entitlement. Record an exact blocker where unavailable; never describe an unperformed test as passed.

Folder contents

- mock-data.json: synthetic input with no real personal data or credentials.
- analyse.py: runnable analysis for this lab only.
- expected-results.json: expected baseline and source checksum.
- ai-review-prompt.txt: evidence-constrained AI review prompt.
- evidence-template.md: your deliverable template.
- INSTRUCTIONS.md and INSTRUCTIONS.pdf: the same procedure in two formats.

1 Prepare a working copy

Copy this whole folder to a location you can edit. Open a terminal in the copied folder. Confirm Python and inspect the data before running code.

```
python3 --version
python3 -m json.tool mock-data.json
```

Identify the record IDs and explain the meaning of each field. All values are invented classroom fixtures. Open analyse.py in the editor and locate the decision rule. It reads a local JSON file and writes a local report.

2 Run the baseline

```
python3 analyse.py --input mock-data.json --output results.json
```

Open results.json. The expected decisions in output order are: ALERT, NO_ALERT. The source_sha256 binds the report to the exact input bytes. Compare against the supplied baseline:

```
python3 -c "import json; a=json.load(open('results.json')); b=json.load(open('expected-
results.json')); assert a==b; print('BASELINE PASS')"
```

A pass proves this fixture was processed as specified; it does not prove an Azure control was deployed. Copy the input checksum and decision rows into your evidence template.

3 Trace the mechanism

Log schema and time

Normalise event time and actor identifiers before correlating records.

Contract: event_id, event_time_utc, user_id, operation, result

Accepted case: Two sources use UTC and the same stable actor ID.

Counterexample: Local time is interpreted as UTC and changes event ordering.

Diagnosis: Retain raw time and normalisation rules; distinguish ingestion delay from event time.

Evidence: Record timezone conversion and the event-to-ingestion delay.

Detection threshold

Group failures by actor and time window; compare the count to a defined threshold.

Contract: window_minutes=5, failed_count>=5

Accepted case: Six failures for user_004 in five minutes trigger review.

Counterexample: Six failures spread across six hours do not meet the rule.

Diagnosis: Use time-bounded grouping and validate service-account baselines to reduce noise.

Evidence: Retain the contributing event IDs and precise window boundaries.

Detection metrics

Compare detector output to labelled cases to measure precision and recall.

Contract: precision=TP/(TP+FP); recall=TP/(TP+FN)

Accepted case: TP=8, FP=2, FN=4 gives 80% precision and 66.7% recall.

Counterexample: Reporting accuracy alone hides missed incidents in an imbalanced set.

Diagnosis: Review false negatives and tune using a held-out dataset; avoid testing on tuning data.

Evidence: Save confusion counts, threshold and dataset version.

4 Change one input and test the consequence

Save a copy of mock-data.json as changed-data.json using the editor. Move e5 to minute 10. The five-minute peak becomes 4 and the alert disappears. Explain the blind spot and propose a separately tested longer-window detector.

```
python3 analyse.py --input changed-data.json --output changed-results.json
```

Compare decisions and source hashes with the baseline. Identify the exact expression in analyse.py responsible for the changed behaviour. Do not overwrite expected-results.json to make a failing baseline pass. Restore the original fixture if you edited it accidentally.

5 Review AI claims against evidence

Open ai-review-prompt.txt. In an organisation-approved AI tool, submit the prompt with the synthetic input and result only. Record the tool/model name, date and prompt version if available. If no approved AI tool is available, manually draft a tentative analyst summary and label it as a human exercise.

For each returned claim, record its cited ID, the actual field value, whether the claim follows, and any correction. Reject conclusions unsupported by the records even when the model is confident. Include one alternative explanation. The AI response is a proposal; it has no authority to approve or execute a security action.

6 Verify the corresponding operational control

Azure procedure H Log Analytics and Sentinel detection

1. Azure Portal > Log Analytics workspaces > Create: use the assigned subscription, rg-itsec-<initials>, approved region and a unique workspace name. Do this only within the trainer's cost allocation.
2. Open Microsoft Sentinel in the supported portal used by the training tenant. Add the allocated workspace where permitted. Record the workspace/resource ID and onboarding result.
3. Open Content hub and locate the Azure Activity solution if required by the tenant. Install or review the approved solution. Open its data connector instructions and use the supported policy-based connection method for the assigned subscription only.
4. Have the authorised trainer configure the required Azure Activity data-collection policy/rule if your role cannot do so. Record the assignment and workspace destination. Do not claim ingestion until an event is observed.
5. In Logs run AzureActivity | take 10. Record available fields and TimeGenerated. If the table is absent, inspect the connector, data destination and permissions; document the blocker.
6. Run AzureActivity | summarize Events=count() by bin(TimeGenerated, 5m), OperationNameValue. Record the time filter and output. This is an activity-volume query, not the synthetic failed-sign-in detector; do not label its count as failed sign-ins.
7. Create or review a scheduled analytics rule using a trainer-approved query and actual available schema. Record frequency, lookback, threshold, entity mapping and incident settings. Keep automated response disabled until separately approved.
8. Inspect a training incident's alerts and entities, then follow Lab 12 to build the timeline and Lab 16 to review response and recovery. Record the underlying events rather than relying only on an AI summary.
9. Review automation rules and a Logic Apps playbook. Identify the trigger identity, allowed actions and approval gate. Do not execute production containment. Disable/remove lab-created rules and workspaces only after the trainer approves cleanup.

For every screenshot or export, record resource scope, UTC time and expected versus observed result. Redact personal data and secrets. If blocked, record the exact error, missing permission/licence, what could be inspected, and the unverified outcome. Ask the trainer to provide an authorised sandbox demonstration where the assessed ability cannot otherwise be evidenced.

7 Submit evidence and clean up

Complete evidence-template.md with baseline, changed result, AI claim review and Azure evidence or clearly labelled limitations. Keep results.json and changed-results.json with your submission. Check that your conclusion distinguishes an observed fact from a hypothesis. Remove only resources or assignments you created with trainer approval; do not delete shared training infrastructure.

Acceptance checks

- The unchanged fixture produces the exact expected report.
- The changed fixture produces the explained result and a different input hash.
- At least one AI or analyst claim is checked against a specific record and field.
- The report states simulation, Azure verified or blocked for each relevant claim.
- The output is a time-window detection output and tuning note with an owner, evidence and remaining uncertainty.

Troubleshooting

- Python not found: install the trainer-approved Python distribution or use py -3 on Windows.

- File not found: open the terminal in this lab folder; check the input filename.
- JSON parse error: validate with python3 -m json.tool changed-data.json; use lowercase true and false in JSON.
- Baseline mismatch: restore the supplied fixture and inspect the first differing decision; never edit the expected file.
- Azure denial or missing feature: capture the exact error and scope. A blocker is a limitation, not proof of competence or deployment.
- AI invents evidence: reject the claim, cite the missing ID, and request an evidence-limited revision.

References

The Learner Guide contains the complete source register and the lecture explanations for this lab. Original examples are adapted from the supplied Azure and AI-security references. Product behaviour should be checked against current Microsoft Learn documentation before a real deployment.